

Al-Nahrain University
College of Engineering
Department of Computer
Fourth Stage/First Semester



Microcontrollers Lab

Experiment No.2 : Bit Toggling of P1 Port on 80C51 Microcontroller

Course Lecture : Mustafa Adnan

Laboratory Assistant: Sarah Haider

Name of student: Aisha Ammar Abdalrhman

Date: 2025/10/30

Objective:

To create a C program that toggles the bits of Port 1 (P1) of the 80C51 microcontroller continuously every 250 milliseconds.

Materials Required:

- 80C51 microcontroller
- Development board (with P1 connected to LEDs or a display)
- Keil C51 or any compatible C compiler
- USB to serial converter (for programming)
- Power supply

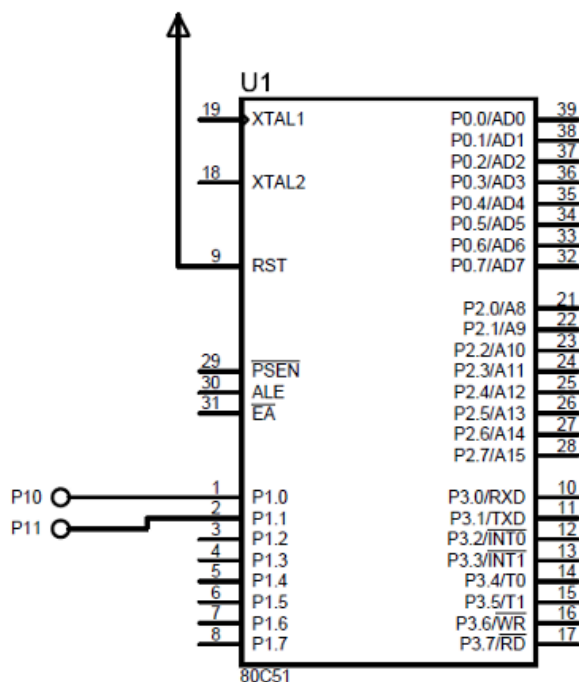
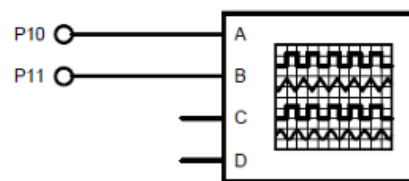
Theory:

The 80C51 microcontroller has four parallel I/O ports (P0, P1, P2, and P3). By toggling the bits of P1, we can control multiple output devices like LEDs. This experiment will help understand how to control hardware using software and the timing operations involved in microcontroller programming.

Procedure:

1. Hardware Setup:

- The LEDs were connected to the P1 port of the microcontroller.
- The microcontroller was ensured to be properly powered and was connected to the development environment.
- The oscilloscope output was drawn.
- The exact waveform timing was found.



2. Writing the C Program:

- The C development environment was opened, and a new project was created.
- The following code was written to toggle the bits of P1.

```
1 #include <reg51.h>
2 #include <stdio.h>
3 void MSDelay(unsigned int);
4 void main(void)
5 {
6     while (1) // repeat forever
7     {
8         P1 = 0x55;
9         MSDelay(250);
10        P1 = 0xAA;
11        MSDelay(250);
12    }
13 }
14 void MSDelay(unsigned int itime)
15 {
16     unsigned int i, j;
17     for (i = 0; i < itime; i++)
18         for (j = 0; j < 1275; j++)
19             ;
20 }
```

3. Compilation of the Program:

- The program was compiled to check for syntax errors.

4. Uploading the Program:

- The USB to serial converter was used to upload the compiled program to the 80C51 microcontroller.

5. Running the Program:

- The microcontroller was powered on, and the LEDs connected to P1 were observed.
- They were seen to turn ON and OFF continuously every 250 milliseconds.

Observations:

- The LEDs connected to Port 1 should blink with a frequency of 2 Hz (ON for 250 ms and OFF for 250 ms).

Questions for Discussion:

1. What is the purpose of the delay function in the program?

- The purpose of the delay function is to create a time interval between each toggle of the bits in Port 1 (P1). This delay allows the LED blinking to be visible to the human eye instead of changing too rapidly.

2. How does changing the inner loop count in the delay function affect the blinking speed?

- Changing the inner loop count alters the duration of the delay.

- Increasing the loop count increases the delay, causing the LEDs to blink more slowly.
- Decreasing the loop count reduces the delay, making the LEDs blink faster.

3. What modifications would be needed to toggle only specific bits of P1 instead of all bits?

- To toggle only specific bits, bitwise operations should be used. For example, use statements like:

```
P1 ^= 0x01; // Toggles only bit 0 of P1
```

This ensures that only certain bits are affected while others remain unchanged.

4. How does the microcontroller handle the toggling of bits in a continuous loop?

- The microcontroller executes the loop instructions repeatedly. Each time the loop runs, it changes (toggles) the state of the bits in P1, and then the delay function pauses execution before the next toggle, creating a continuous ON–OFF cycle.

5. What practical applications can you think of for toggling bits in a microcontroller?

Toggling bits can be used in many applications, such as:

- Blinking LEDs for status indication
- Generating clock or timing signals
- Controlling relays or motors
- Creating square waves for signal testing
- Turning devices ON and OFF periodically

6. What would happen if the delay function was removed from the program?

- If the delay function was removed, the LEDs would toggle too quickly for the human eye to see. It would appear as though the LEDs are constantly ON or dimly glowing because the toggling happens thousands of times per second.

7. How can you modify the program to toggle P1 at different rates, such as 500 ms?

- The toggling rate can be changed by adjusting the delay value.

For example, if the delay function currently creates a 250 ms delay, doubling the loop count or delay time constant would produce a 500 ms toggle rate.

8. What potential issues could arise from using an infinite loop in microcontroller programming?

Using an infinite loop may cause:

- High power consumption if the microcontroller runs continuously.
- Difficulty in stopping or resetting the program without external control.
- Processor blocking, preventing other tasks or interrupts from running if not properly managed.

9. How could you implement a user-controlled toggle rate using a switch or button?

- A switch or button could be connected to an input pin, and its state can be read in the program. Based on the button press, different delay values could be applied. For example:

- One button press sets the delay to 250 ms.
- Another press changes it to 500 ms.

This allows the user to control the blinking speed dynamically.

10. In what ways can bit manipulation techniques improve efficiency in microcontroller programming?

-Bit manipulation techniques enhance efficiency by enabling faster execution, reduced memory usage, and precise hardware control. They minimize code size, improve real-time performance, and lower power consumption by allowing direct access to specific bits within registers or ports. This leads to more compact, responsive, and energy-efficient microcontroller programs.

Conclusion:

This experiment demonstrates the fundamental concepts of I/O operations, timing, and continuous control in embedded systems using the 80C51 microcontroller. By toggling the bits of Port 1, we learned how to control hardware through software effectively.

